Advanced Machine Learning

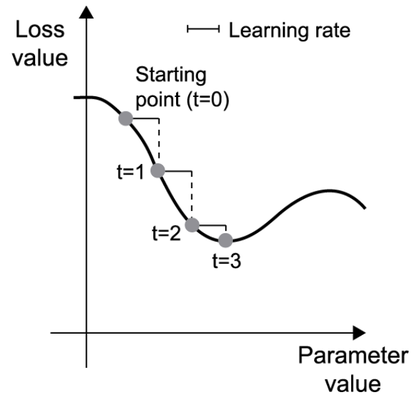
DATA 642

Lecture 2 — Methods for Unconstrained Optimization

*Figures and notes are from the book Mathematics for Machine Learning by A. Aldo Faisal, Cheng Soon Ong, and Marc Peter Deisenroth

Basic idea through an example for line search methods for one variable

- Draw training samples, $\boldsymbol{x}$, and corresponding targets, y_{true}.
- Utilize the model to predict outcomes, y_{pred}, based on the input data $\boldsymbol{x}$.
- Calculate the loss function, which quantifies the mismatch between predicted y_{pred} and true y_{true} values.
- Compute the gradient of the loss with regard to the model's parameters.
- Adjust the model parameters in the direction opposite to the gradient, thereby reducing the loss on the training dataset.



Line search methods in a formal manner

Each iteration of a line search method computes a search direction $\mathbf{s}^{(k)}$ and then decides how far to move along that direction. The iteration is given by

$$\boldsymbol{\theta}^{(k+1)} = \boldsymbol{\theta}^{(k)} + \alpha^{(k)}\mathbf{s}^{(k)}.$$

The main idea of the procedure is the following

1. Determine a search direction $\mathbf{s}^{(k)}$. At each iteration k, a search direction $\mathbf{s}^{(k)}$ is computed. This direction typically depends on the optimization algorithm being used.

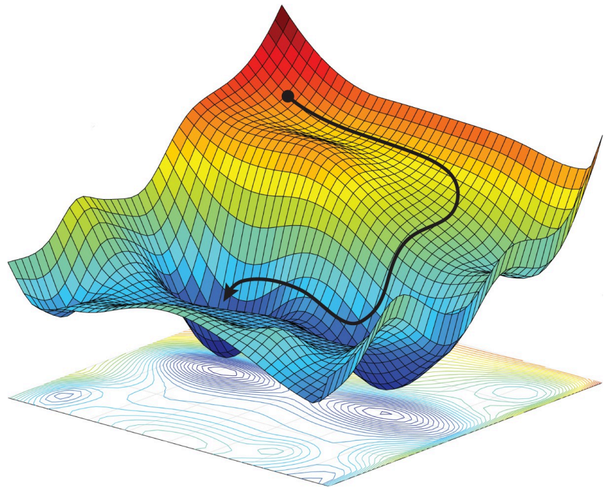
2. Find $\alpha^{(k)}$ to minimize $J(\boldsymbol{\theta}^{(k)} + \alpha\mathbf{s}^{(k)})$ w.r.t α. Once the search direction is determined, the next step is to find the appropriate step size $\alpha^{(k)}$ that minimizes the objective function $J(\boldsymbol{\theta}^{(k)} + \alpha\mathbf{s}^{(k)})$ with respect to α. This step ensures that the objective function decreases sufficiently along the chosen direction. Step size is also called learning rate.

3. Set $\boldsymbol{\theta}^{(k+1)} = \boldsymbol{\theta}^{(k)} + \alpha^{(k)}\mathbf{s}^{(k)}$. Finally, the parameters $\boldsymbol{\theta}^{(k)}$ are updated using the computed step size $\alpha^{(k)}$ and the search direction $\mathbf{s}^{(k)}$.

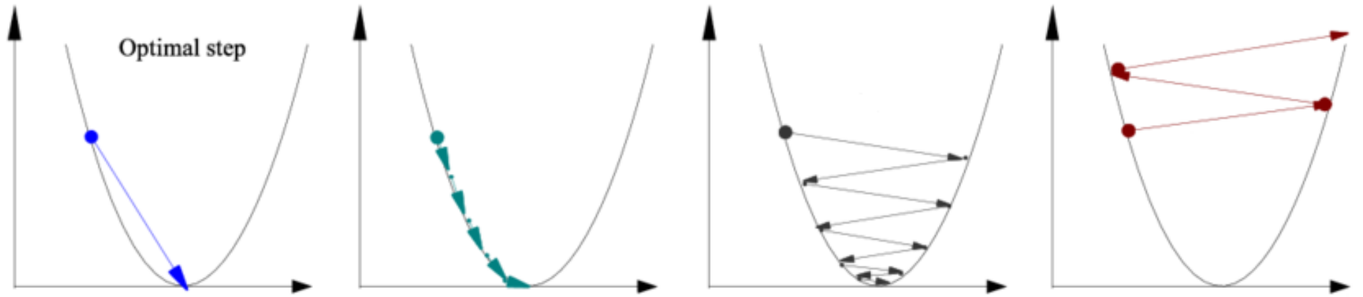
But which direction to search? The direction $\mathbf{s}^{(k)}$ should satisfy the **descent property**:

$$\mathbf{s}^{(k)^\top} \nabla J\left(\boldsymbol{\theta}^{(k)}\right) < 0.$$

If the choice $\mathbf{s}^{(k)} = -\nabla J\left(\boldsymbol{\theta}^{(k)}\right)$ is made, the resulting line search is known as the **steepest or gradient descent method**.



Although steepest descent sounds effective, it can be very inefficient. For instance, the figure below illustrates the impact of the step size in convergence. Its asymptotic rate of convergence is inferior to many other methods. Using the ball rolling down the hill analogy, when the surface is a long, thin valley, the problem is poorly conditioned (Trefethen and Bau III, 1997). For poorly conditioned convex problems, gradient descent increasingly "zigzags" as the gradients point nearly orthogonally to the shortest direction to a minimum point



Example

We can learn much about the steepest descent method and see difficulties by considering the simple example, in which the objective function is quadratic and the line searches are exact. Let

$$J(\boldsymbol{\theta}) = \frac{1}{2}\boldsymbol{\theta}^\top \mathbf{Q}\boldsymbol{\theta} - C, \tag{1}$$

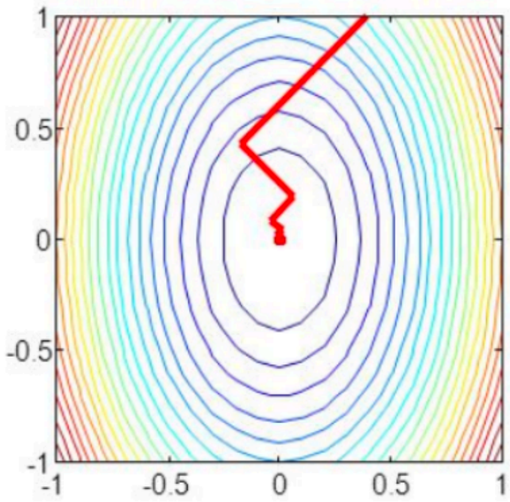
where $\mathbf{Q}$ is symmetric positive definite and C is a constant. Since $\mathbf{Q}$ is symmetric there exists an orthogonal matrix $\mathbf{U}$ such that $\mathbf{U}^\top \mathbf{Q}\mathbf{U}$ is a diagonal matrix $\boldsymbol{\Lambda}$. The main diagonal of $\boldsymbol{\Lambda}$ are the eigenvalues of $\mathbf{Q}$, which are positive. Introducing new coordinates $\mathbf{z} = \mathbf{U}^\top \boldsymbol{\theta}$ and by dropping the constant C, the minimization of $J(\boldsymbol{\theta})$ is equivalent to minimizing

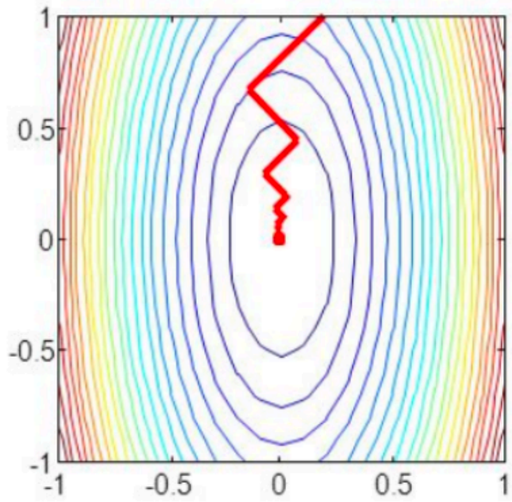
$$J(\mathbf{z}) = \sum_{i=1}^{n} \lambda_i z_i^2. \tag{2}$$

So if we consider the simple case where our matrix $\mathbf{Q}$ is 2×2 the contours have the form

$$\lambda_1 z_1^2 + \lambda_2 z_2^2 = c,$$

which are ellipses centered at the origin. We conclude that the shape of the contours depends on the condition number of $\mathbf{Q}$, so the convergence rate of steepest algorithm really depends on $\mathbf{Q}$.





- The convergence of gradient descent may be very slow if the curvature of the optimization surface is such that there are regions that are poorly scaled. The curvature is such that the gradient descent steps hops between the walls of the valley and approaches the optimum in small steps.

Gradient descent with momentum

Convergence Challenges in Gradient Descent:

Slow convergence can occur when the optimization surface has regions poorly scaled. The curvature of the surface causes gradient descent steps to oscillate between valley walls, leading to small steps towards the optimum.

Introduction to Gradient Descent with Momentum:

Gradient descent with momentum, proposed by Rumelhart et al. (1986), addresses slow convergence by introducing a memory term. This memory term remembers previous iterations, dampening oscillations and smoothing gradient updates.

Analogy with a Heavy Ball:

In the analogy, the momentum term resembles a heavy ball reluctant to change directions, leading to more stable optimization.

Implementation Details:

Gradient update with memory involves a moving average, where the next update is determined as a linear combination of current and previous gradients.

$$\boldsymbol{\theta}^{(k+1)} = \boldsymbol{\theta}^{(k)} - \alpha^{(k)} \nabla J(\boldsymbol{\theta}^{(k)}) + \beta \Delta \boldsymbol{\theta}^{(k)} \tag{3}$$

$$\Delta \boldsymbol{\theta}^{(k)} = \boldsymbol{\theta}^{(k)} - \boldsymbol{\theta}^{(k-1)} = \beta \Delta \boldsymbol{\theta}^{(k-1)} - \alpha^{(k-1)} \nabla J(\boldsymbol{\theta}^{(k-1)}) \tag{4}$$

where α is the learning rate and β is the momentum parameter, in the range [0, 1].

Handling Noisy Gradient Estimates: The momentum term is beneficial when dealing with noisy gradient estimates, as it averages out different noisy estimates of the gradient.

Stochastic Approximation: Stochastic approximation methods can provide approximate gradients, which can be effectively handled by gradient descent with momentum.

Stochastic Gradient Descent (SGD)

Stochastic gradient descent (SGD) is a stochastic approximation of the steepest descent method used to minimize objective functions composed of the sum of differentiable functions.

- **Stochastic Nature**:
    - SGD acknowledges that we only have a noisy approximation of the gradient rather than the exact gradient.
- **Objective Function Formulation**:
    - For N data points, the objective function is often expressed as the sum of individual sample losses:

$$J(\boldsymbol{\theta}) = \sum_{n=1}^{N} J_n(\boldsymbol{\theta}) \tag{5}$$

- **Batch Optimization vs. SGD**:
    - Standard gradient descent is a "batch" optimization method that uses the full training set for optimization.
    - Computing gradients for each sample can be computationally expensive, especially for large datasets with no simple formulas.
- **Reducing Computational Complexity**:
    - SGD reduces computational complexity by computing gradients using a subset of samples, rather than the entire dataset.
    - The sum of gradients from a subset serves as an empirical estimate of the expected value of the gradient.
- **Convergence**:
    - With an appropriate learning rate schedule, SGD converges almost surely to a local minimum, under mild assumptions.

Mini-batch gradient descent

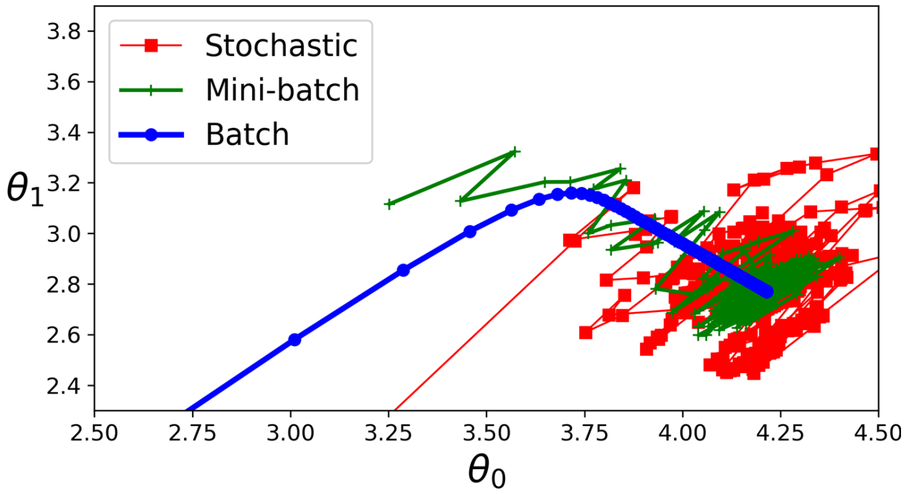
- Mini-batch gradient descent is a variation of stochastic gradient descent (SGD) where instead of computing the gradient using a single data point (as in SGD), we randomly select a subset (mini-batch) of the training data to estimate the gradient.

- This approach strikes a balance between the computational efficiency of stochastic gradient descent and the stability of standard gradient descent, making it suitable for large datasets and complex models.

- By taking a subset of data for gradient estimation, mini-batch gradient descent still provides an unbiased estimate of the true gradient, ensuring convergence to a local minimum with appropriate learning rate scheduling.

Convergence Trajectories in Optimization:

- Standard gradient descent shows smooth and steady convergence but may be computationally expensive for large datasets.
- Stochastic gradient descent exhibits more erratic convergence due to the noisy nature of individual gradient estimates but is computationally efficient.
- Mini-batch gradient descent strikes a balance between stability and efficiency by using a subset of data for gradient estimation, resulting in smoother convergence compared to SGD.



Why should one consider using an approximate gradient?

Practical Implementation Constraints:

- One reason for using mini-batch gradient descent is practical limitations, such as CPU/GPU memory constraints and computational time limits.
- Mini-batch size is akin to sample size when estimating empirical means.

Advantages of Large Mini-Batches:

- Large mini-batch sizes provide accurate gradient estimates, reducing parameter update variance.
- They leverage optimized matrix operations for efficient computation.
- More stable convergence is achieved, but each gradient calculation becomes more expensive.

Advantages of Small Mini-Batches:

- Small mini-batches allow for quick gradient estimation.
- The noise in gradient estimates helps escape bad local optima.
- In machine learning, approximate gradients are often sufficient for training to improve generalization performance.